EvoCracker

AI RESEARCH PROJECT

Phase 1 of 5 — Core System Documentation

PHASE 1

Core System Documentation

State space, agent capabilities & fitness function formulation

Deliverable Summary

- Defined the formal state space and runtime agent action space.
- Documented movement topology, node-level pathfinding state, and behavior-state transitions.
- Captured the baseline fitness formulation and adaptive playstyle multiplier logic.

Verification Checklist

✓ State model documented and mapped to in-project structures.
✓ Agent sensory/action capabilities listed in implementation terms.
✓ Fitness formulation documented with adaptive adjustments.

1. Game State Space

The game's state space represents the environment the AI agents navigate and the data structures that encode their current situation.

Grid-Based Environment

The world is represented as a 2D discrete grid (Grid). Each cell (GridNode) has coordinates (x, y) and contains:

- walkable — Boolean indicating if the cell can be traversed.
- weight — Movement cost derived from terrain type.
- tileType — Physical property: FLOOR_STONE, WALL, WATER, MUD, TRAP.

Navigational State

During pathfinding, each node temporarily stores search state:

- g — Exact cost from the start node.
- h — Heuristic estimated cost to the goal.
- f — Total estimated cost (g + h).
- visited, inOpenSet, depth, and parent.

Connectivity

The environment supports 4-directional (orthogonal) and 8-directional (orthogonal + diagonal) movement. Diagonal movements are cost-adjusted (√2) and prevent corner-cutting through walls.

Behavioral States

An agent's behavior is governed by a Behavior Tree transitioning between:

IDLE	PATROL	INVESTIGATE	CHASE	FLEE

2. Agent Capabilities

Agents possess a wide array of capabilities driven by their behavior trees, genetic traits, and pathfinding systems.

2.1 Sensory Capabilities

FOV	Agents utilize a raycasting-based Field of View (FOV) to detect the player and other dynamic objects within their line of sight.

2.2 Action Capabilities

- Movement — Pathfinding towards a specific target or exploring via patrol movements.
- Combat — Dealing damage to the player upon successful engagement.
- Cooperation — Assisting other agents to secure cooperative kills or flanking maneuvers.
- Fleeing — Retreating when at a disadvantage or executing hit-and-run tactics.

2.3 Search Algorithms

Each enemy type has a hardcoded default algorithm assigned at spawn. This is the active mechanism in production. An algorithmWeights gene was implemented and wired into the GA mutation pipeline, but after testing it caused the population to converge onto 1–2 dominant algorithms within a few generations, making the Pathfinding dashboard comparison meaningless. The default mapping was introduced to fix this — each enemy type permanently runs a specific algorithm, guaranteeing all 8 are visible and comparable at all times.

Algorithm	Description
`BFS`	Complete, unweighted breadth-first search.
`DFS`	Depth-first traversal — exploratory, non-optimal pursuit.
`IDS`	Iterative deepening search combining DFS space efficiency with BFS completeness.
`DLS`	Depth-limited search with genome-driven depth limits.
`UCS`	Uniform-cost search weighted by tile movement costs.
`A*`	Heuristic pathfinding using g + h — optimal and efficient.
`Greedy BFS`	Heuristic-only best-first search — fast but not always optimal.
`Hill Climbing`	Local heuristic improvement — fast but incomplete movement.

2.4 Genetic Traits (Genome)

Each agent possesses a Genome of values between 0.0 and 1.0 governing its capabilities:

Gene / Trait	Description
`speed`	Determines the agent's movement velocity.
`vision`	Modifies the range and arc of the agent's FOV.
`aggression`	Likelihood to attack versus flee.
`persistence`	How long the agent chases after losing line of sight.
`cautiousness`	Hesitation before engaging or entering unknown areas.
`packTendency`	Desire to group up with other agents.
`ambushTendency`	Preference for hiding near choke points.
`patrolVariance`	Degree of randomness in patrol routes.
`algorithmWeights`	Designed to evolve weighted algorithm preferences — implemented but superseded in production by per-enemy default algorithm assignment (see Section 2.3).

3. Mathematical Formulation of the Fitness Function

The Genetic Algorithm evolves the enemy population between floors. The fitness function rewards agents that successfully challenge the player's specific playstyle.

3.1 Base Fitness Calculation

The base fitness is a weighted sum of performance metrics gathered during the agent's lifetime:

Metric	Weight	Description
Time Visible	× 0.30	Seconds the player spent in agent's FOV
Damage Dealt	× 2.00	Total damage dealt to the player
Detections	× 1.50	Times a hidden player was successfully spotted
Survival Time	× 0.20	Seconds the agent stayed alive
Area Covered	× 0.40	Unique tiles explored by the agent
Time Stuck	× −1.00	Seconds unable to move — penalized
Cooperative Kills	× 2.50	Kills/heavy damage assisted by other agents

```
Base Fitness = (TimeVisible × 0.3) + (Damage × 2.0) + (Detections × 1.5) +
(Survival × 0.2) + (Area × 0.4) − (Stuck × 1.0) + (CoopKills × 2.5)
```

3.2 Playstyle-Adaptive Bias

A player profile is constructed (rusher, stayer, explorer, fighter, or hybrid). Base fitness is then scaled based on the agent's traits that best counter the player:

Playstyle	Adaptive Multiplier	Counter Strategy
Stayer	`Persistence × 0.5`	Counters players who hide frequently
Rusher	`Speed × 0.5`	Counters players who sprint to the exit
Explorer	`Pack Tendency × 0.4`	Counters players who visit every room
Fighter	`Cautiousness × 0.3`	Counters players who engage often
Hybrid	`× 1.1 (flat)`	Balanced boost when no dominant pattern

```
Final Fitness = Math.max(0, Final Fitness)  // Clamped to minimum of 0
```